Lab 08 — Configuration and secrets

Theory — how a single image serves development, staging and production without being rebuilt, and why a password that enters an image, a command line or a log never comes back out.

Objectives

- Understand why configuration must live **outside the image**, and what the *12-factor* principle says.
- Master `ENV`, `-e`, `--env-file`, their order of precedence and the `_FILE` convention.
- Connect environment variables to Spring Boot configuration (*relaxed binding*, profiles).
- Know **where a secret can be seen** (image, `inspect`, `exec`, logs, `ps`) and therefore where it must not go.
- Use `podman secret` at run time and `--mount=type=secret` at build time.

---

1. One image, three environments

In lab 05 you built `api:1.0`. That image has to run on your workstation against a local database, on the staging server against a test database, and in production against the real one — with different URLs, passwords and log levels. If each environment had its own image, staging would be testing an image **that is not** the one going to production. The whole discipline of deployment rests on the opposite: **one immutable image, promoted from one environment to the next, with configuration injected from outside**.

> **→ HISTORY**
> In 2011, Heroku's engineers published *The Twelve-Factor App*, twelve rules for cloud-deployable applications. Factor III, *Config*, is the most quoted: "store config in the environment", because environment variables are independent of language, operating system and configuration file, and cannot be committed to Git by accident. Docker (2013) made it the default practice: `-e`.

What changes between environments is **configuration**: database URL, credentials, ports, log levels, feature flags. What does not change is **code**, and it lives in the image. And within configuration, whatever grants access to something is a **secret**: password, API key, private certificate. The three are not handled the same way.

2. The mechanisms

Mechanism	Where	Scope	Use
`ENV` in the Dockerfile	in the image	every container of the image	a sensible **default value**, never a secret
`-e KEY=value`	`podman run`	this container	override one or two values
`--env-file api.env`	`podman run`	this container	one set of values per environment, file kept out of Git
mounted file (`-v ./application.yaml:/config/…:ro`)	`podman run`	this container	structured, long, versioned configuration
`--secret` (`/run/secrets/…`)	`podman run`	this container	passwords, keys, certificates
command argument	`podman run … --password=x`	this container	**never** for a secret (visible in `ps` , `logs` , `inspect`)

The precedence is fixed: **`-e` overrides `--env-file`, which overrides the image's `ENV`**, whatever the order of the options on the line. And `-e KEY` with no value passes the variable **from the host shell**: handy in CI, a trap during review.

> **→ LINUX**
> Environment variables are a block of `KEY=value` text that every process receives from its parent at creation, and that every child inherits. They are neither encrypted nor protected: `/proc/<pid>/environ` exposes them to any process of the same user, and `podman exec container env` prints them. A program cannot "forget" them once started — which is why a secret in a variable is **readable**, but also why it is so simple.

> **→ SPRING BOOT**
> Spring Boot reads **all** its properties from the environment thanks to *relaxed binding*: `spring.datasource.url` becomes `SPRING_DATASOURCE_URL` , `server.port` becomes `SERVER_PORT` , and `SPRING_PROFILES_ACTIVE=prod` activates the `prod` profile (hence `application-prod.yaml`). A file mounted at `/config/application.yaml` is read automatically, and `SPRING_CONFIG_ADDITIONAL_LOCATION` points to others. The order: environment variable > external file > file inside the JAR. Exactly the container engine's logic, one floor up.

> **→ JAVA**
> The JVM reads a variable too: `JAVA_TOOL_OPTIONS` . Its content is appended to the start-up options, and the JVM reports it with `Picked up JAVA_TOOL_OPTIONS: …` on standard error. That is how you tune memory (`-Xmx` , `-XX:MaxRAMPercentage`) or attach an agent without touching the image's `ENTRYPOINT` — lab 10 uses it for memory limits.

3. Where a secret can be seen

Take the inventory before choosing a mechanism. A secret placed…

- **in the image** (`ENV` , `COPY .env` , `ARG` — lab 04) is readable through `podman history` , `podman image inspect` , `podman run --rm image cat /app/.env` , and by anyone who pulls the image from the registry. **Never.**

- **as a command argument** (`--db-password=x`) shows up in `podman inspect` (`Config.Cmd`), `podman top`, the host's `ps`, and often in the logs of an application that prints its arguments. **Never.**
- **as an environment variable** (`-e`, `--env-file`) shows up in `podman inspect` (`Config.Env`), `podman exec env`, `/proc/1/environ`, and in any error report or *dump* that includes the environment. **Acceptable** on a workstation or in CI, **insufficient** in production.
- **in a read-only mounted file**, or an engine secret, appears neither in `inspect` nor in the environment; only the process that reads the file sees it. That is the target.

> **PITFALL**
> Almost every official image offers the `_FILE` convention: `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` makes it read the password from a file instead of `POSTGRES_PASSWORD`. The environment then holds only a **path**. Check that your own application can do the same: Spring Boot does it with `spring.config.import=optional:configtree:/run/secrets/`, which turns every file in the directory into a property.

4. Engine secrets

`podman secret create db_password -` reads a secret from standard input and stores it in your user's storage; `podman secret ls` lists it, `podman secret inspect` shows its metadata without the value, `podman secret rm` deletes it. At `run`, `--secret db_password` mounts it inside the container at `/run/secrets/db_password`, read-only, and **nothing appears in `podman inspect`**. The variant `--secret db_password,type=env,target=POSTGRES_PASSWORD` injects it as an environment variable for applications that do not read files — visible through `exec env`, but still absent from `inspect`.

> **PODMAN**
> `podman secret` works without any orchestrator. With Docker, `docker secret` requires Swarm mode; outside Swarm, you fall back on Compose's `secrets:`, which are plain mounted files — and which `podman compose` understands too. As for storage, Podman's default driver (`file`) keeps the value **encoded, not encrypted**, in `~/.local/share/containers/storage/secrets/`, and `podman secret inspect --showsecret` reads it back: it is a drawer, not a safe. Quadlet (lab 10) can reference a secret there, and Kubernetes has its `Secret` object (lab 14), with the same limitation.

> **→ SECURITY**
> In the workplace, secrets live in a **secrets manager** (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault): encrypted at rest, access logged, automatic **rotation**. The container receives either a file injected at start-up, or a short-lived token to fetch the secret itself. The goal is not "nobody can read it"; it is "we know who read it, and we can change it in a minute".

5. Build-time secrets

An `npm install` or `mvn package` sometimes needs a token for a private repository, **during** the build only. Lab 04 showed that `ARG` ends up in `podman history`. The right answer is the secret mount: `RUN --mount=type=secret,id=token,required=true cat /run/secrets/token` makes the file available for **that single instruction**, and `podman build --secret id=token,src=token.txt` supplies it. It is written to no layer, `history` only shows the `--mount=…` line, and `/run/secrets` no longer exists in the image. Without `--secret`, `required=true` fails the build: `secret required but no secret with id "token" found`.

Beware of the cache: a `RUN` instruction already in cache **is not re-executed**, so it does not ask for the secret again. An expired token goes unnoticed until the next `--no-cache`.

6. The `.env` file

One `api.env` (or `.env`) file per environment is the most widespread form: one `KEY=value` line per variable, `#` for comments, no quotes and no spaces around the `=`. Three rules: it is in `.gitignore` **and** in `.dockerignore` (lab 04); an `api.env.example` with empty values is committed in its place; and it holds no production secrets — those go through a secret or a mounted file.

7. In the workplace

- **One image, promoted**: CI builds `api:1.4.2` once, pushes it, and the same digest is deployed to staging and then to production with different variables.
- **Spring profiles by variable**: `SPRING_PROFILES_ACTIVE=prod` in the deployment, never in the Dockerfile; `application-prod.yaml` holds no password but `${DB_PASSWORD}`, resolved from the environment or a secrets file.
- **Secrets**: files injected by the orchestrator or the secrets manager, `_FILE` or `configtree`, scheduled rotation. A secret that shows up in a log, a ticket or an `inspect` is considered **compromised** and changed.
- **Review**: an `ENV *_PASSWORD` or a `COPY .env` in a Dockerfile is rejected in code review, automatically when possible (*linters* such as Hadolint, scanners such as Trivy or Gitleaks).

Remember

- One immutable image, injected configuration: `ENV` for defaults, `-e` / `--env-file` for the environment, files or secrets for anything sensitive.
- Fixed precedence: `-e` > `--env-file` > the image's `ENV`.
- Spring Boot reads everything from the environment (`SPRING_DATASOURCE_URL`, `SPRING_PROFILES_ACTIVE`); the JVM reads `JAVA_TOOL_OPTIONS`.
- A secret as an argument shows in `ps`, `inspect` and the logs; as a variable, in `inspect` and `exec env`; as a mounted file or `--secret`, nowhere except to the process.
- `_FILE` or `configtree`: the environment carries only a path.
- `podman secret` stores without encrypting; the real safe is a secrets manager with rotation.
- At build time: `--mount=type=secret`, never `ARG`; and the cache does not ask for a secret again.

Vocabulary

12-factor: twelve rules for deployable applications, including "config in the environment". — **configuration / secret**: what varies per environment / what grants access. — **`ENV`**: default variable written into the image. — **`--env-file`**: `KEY=value` file loaded at `run`. — **relaxed binding**: Spring Boot's translation of properties into variables. — **Spring profile**: configuration set activated by `SPRING_PROFILES_ACTIVE`. — **`_FILE` convention**: read a value from a file whose path is in a variable. — **`podman secret`**: engine-managed secret, mounted in `/run/secrets/`. — **build secret**: file available during a single `RUN` instruction. — **secrets manager**: encrypted, logged service with rotation.